VELLORE INSTITUTE OF TECHNOLOGY

[School / Department name]

Operating Systems – Course Project

REVIEW 2

Methodology, Implementation and Initial Results

MediQueue: An AI-Assisted Emergency Department Queue Built on Priority Scheduling and Process Synchronization

Submitted by

Name	Register Number
Aahan Anand	24BCE2564
Ayush Anupam Sahu	24BAI0123
Ankit Dhupal	24BCB0142

Under the guidance of

[Faculty Name]

GitHub: https://github.com/anandaahan7-prog/mediqueue

Live demo: https://mediqueue-eight-tan.vercel.app

1. Finalized System Architecture

The final system has three tiers (Fig 1). The screens for the nurse, doctor, bed board, waiting room and admin run in the browser and ask the server for the latest state (/api/state) every 5 seconds. The Flask server runs each request in a separate thread. Because of this, the two OS modules (the dispatcher and the bed manager) can be called by many threads at once and are written as critical sections. All data is kept in SQLite, and the three trained models are loaded from .joblib files when the server starts.

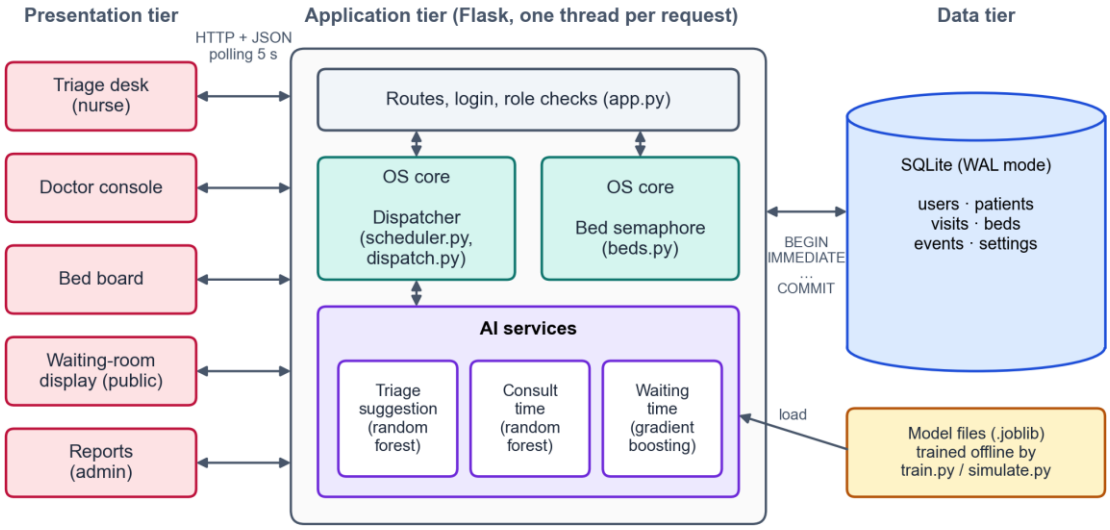


Fig 1: Finalized architecture of MediQueue

Compared to Review 1, the architecture did not change much. We added the waiting-time model to the AI layer, the events table for the audit log, and the deployment on Vercel. The level 1 DFD in Fig 2 shows how data moves between the processes and data stores.

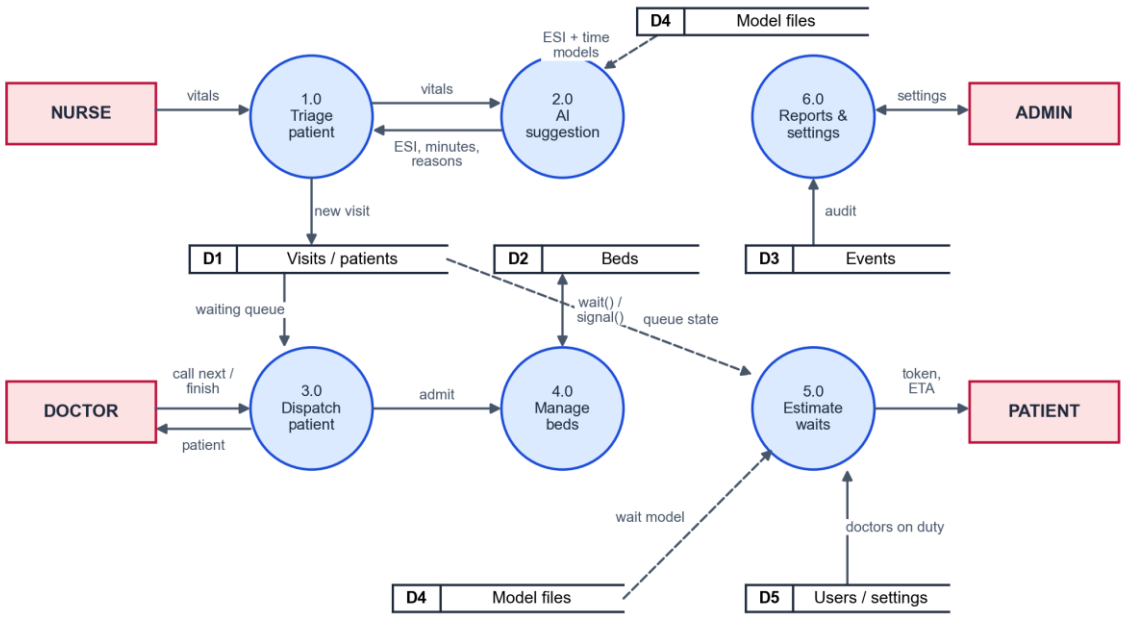


Fig 2: Level 1 data flow diagram

2. Detailed Methodology

We followed the steps below. Each step produced something that was used in the next one.

2.1 Requirement study

We studied the ESI triage handbook [1] and NEWS2 [2] and wrote down every decision that is taken in the emergency department: what level to give at the triage desk, who goes in when a doctor is free, what happens when a critical patient comes while all doctors are busy, and who gets a bed. Only the decisions that are really a scheduling or a synchronization problem were kept for the OS part.

2.2 Dataset creation and model training

Since we did not have access to real hospital data, we wrote a generator for synthetic triage cases (Section 6). Two random forest models were trained on it, one for the triage level and one for the consultation time, and checked with 5-fold cross validation. A safety margin and red-flag rules were added after we saw that the plain model under-triaged about 9% of patients.

2.3 Scheduler design

The dispatch rule is a multilevel priority queue with one level per ESI category. Aging moves a waiting patient up one level every 30 minutes, but only up to level 3. Inside a level patients are served first come first served. Preemption is allowed only for ESI 1. The rule was written as pure Python functions in scheduler.py so that the same code runs in the web app and in the simulator.

2.4 Synchronization design

"Call next patient", admitting a patient and marking a bed clean are run inside a database transaction that starts with BEGIN IMMEDIATE (acquire the write lock) and ends with COMMIT (release it). The updates also re-check the status so that a patient or bed can never be taken twice. The beds are handled like a counting semaphore with the boarding patients as its blocked queue.

2.5 Application development

The web application was built with Flask, SQLite, Jinja2 templates and plain JavaScript. Each role (nurse, doctor, admin) has its own screen, and the waiting room display needs no login.

2.6 Experiments

We ran three kinds of experiments: cross validation of the AI models, a simulation of 40 shifts to compare the scheduling methods, and a multi-threaded stress test to check the synchronization. The setup is in Section 8 and the results are in Section 10.

3. Algorithm / Flowchart

3.1 Proposed scheduling algorithm (MediQueue)

```
Input : waiting patients W, doctor d, aging interval A = 30 min
Output: patient assigned to d

1. BEGIN IMMEDIATE                        // acquire lock
2. if d already has a patient: ROLLBACK, stop
3. for each p in W:
       if esi(p) <= 2 : level(p) = esi(p)
       else           : level(p) = max(3, esi(p) - floor(wait(p) / A))
4. p* = patient with smallest (level, arrival time)
5. UPDATE visit p* to 'in_consult' only if it is still 'waiting'
6. if no row changed: ROLLBACK, stop
7. COMMIT                                 // release lock
8. show token of p* on doctor screen and waiting room display
```

3.2 Preemption for ESI 1

```
if an ESI 1 patient is waiting and every doctor on duty is busy:
    d = doctor whose current patient has the highest ESI number (>= 3)
    if d exists:
        ask d to pause; paused patient keeps arrival time and
        minutes already done, goes back to the waiting queue
        d is given the ESI 1 patient
    else:
        all doctors have ESI 1-2 patients -> ask for backup
```

3.3 Bed allocation (counting semaphore)

```
wait(beds)  - doctor admits patient p
    if free bed exists: give bed, p.status = admitted
    else              : p.status = boarding        (blocked)

signal(beds) - housekeeping marks bed b clean
    if boarding queue not empty: give b to most urgent boarder
    else                       : b.status = free   (value + 1)
```

3.4 Triage suggestion

```
x  = [age, HR, SBP, RR, SpO2, temp, AVPU, pain, complaint one-hot]
P  = RandomForest.predict_proba(x)        // P(ESI = 1..5)
y  = argmax(P)
if y > 1 and P(y - 1) >= 0.30 : y = y - 1  // safety margin
if red flag present           : y = 1      // SpO2 < 85, SBP < 80 ...
reasons = effect of resetting each abnormal input to a normal value
nurse accepts y or changes it with a written reason
```

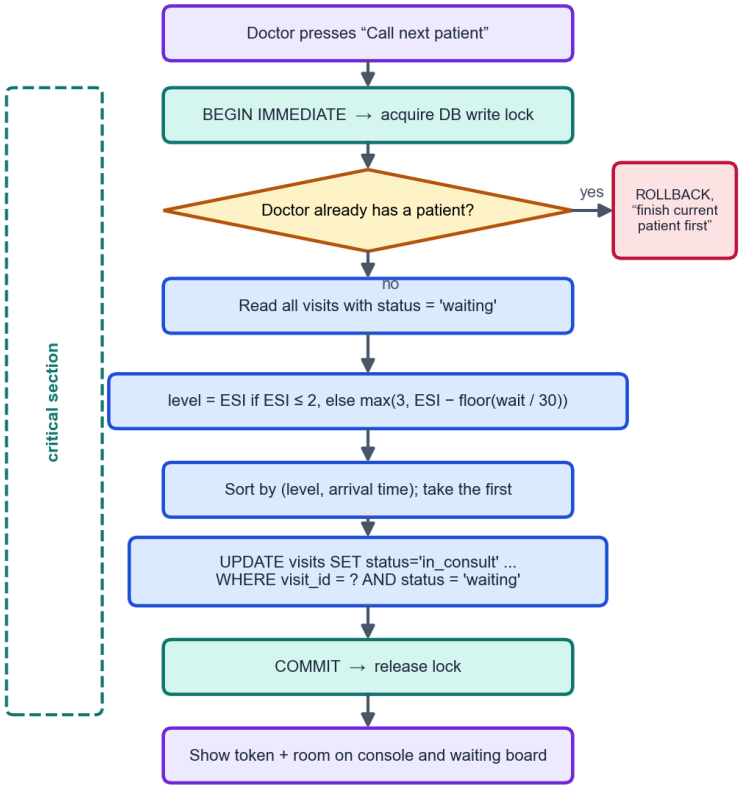


Fig 3: Flowchart of "Call next patient"

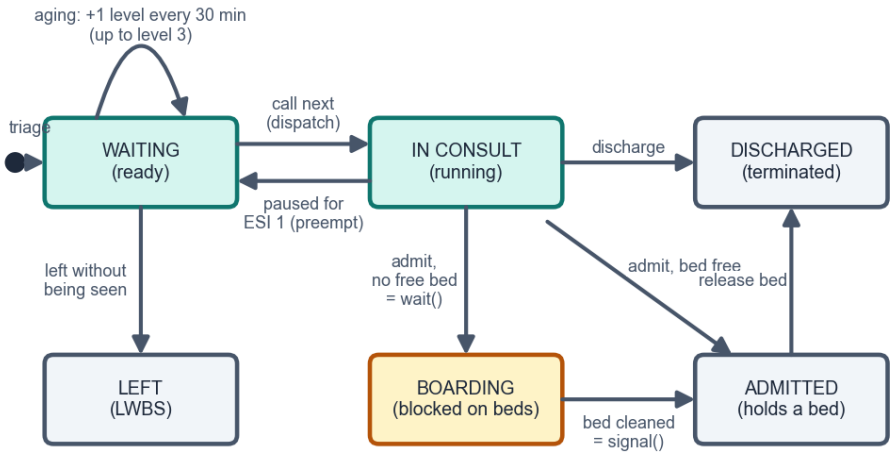


Fig 4: States of a patient visit (same as process states)

4. Module-wise Implementation

Table 1: Modules and files

Module	File(s)	What it does
Scheduler	scheduler.py	Effective level with aging, queue order, preemption target, wait replay
Dispatcher	dispatch.py	Call next patient, pause for ESI 1, discharge /

		admit (critical sections)
Bed manager	beds.py	wait_bed(), release_bed(), signal_bed() on the beds table
Triage model	ai/dataset.py, ai/train.py, ai/triage_model.py	Dataset, training, suggestion with safety rules and reasons
Waiting-time model	ai/wait_model.py, simulate.py	Features from the queue, gradient boosting correction
Database	schema.sql, db.py, seed.py	Tables, constraints, connection settings, demo data
Web application	app.py, templates/, static/	Routes, login, live state API, user interface
Testing	tests/test_system.py, race_test.py	14 functional tests, concurrency stress test

4.1 Scheduler module

The effective level of a waiting patient is calculated from the ESI level and the waiting time. The queue is ordered by (level, arrival time).

```
AGING_MINUTES = 30
AGING_CAP = 3

def effective_level(esi, waited, aging_minutes=AGING_MINUTES):
    if esi <= 2:
        return esi
    return max(AGING_CAP, esi - int(waited // aging_minutes))

def queue_key(visit, now, aging_minutes=AGING_MINUTES):
    waited = now - visit["arrived"]
    level = effective_level(visit["esi"], waited, aging_minutes)
    # smaller key = called first
    return (level, visit["arrived"], visit["id"])
```

4.2 Dispatcher module (critical section)

BEGIN IMMEDIATE takes the SQLite write lock before the queue is read, so a second doctor's request waits until the first one commits. We used the database lock instead of Python's threading.Lock because it also works when the server runs more than one process.

```
def call_next(conn, doctor_id):
    conn.execute("BEGIN IMMEDIATE")        # enter critical section
    try:
        rows = conn.execute(
            "SELECT * FROM visits WHERE status = 'waiting'").fetchall()
        jobs = [as_job(r) for r in rows]
        nxt = pick_next(jobs, now_minutes(), "mediqueue", aging)
        cur = conn.execute(
            "UPDATE visits SET status = 'in_consult', doctor_id = ? "
            "WHERE visit_id = ? AND status = 'waiting'",
            (doctor_id, nxt["id"]))
        if cur.rowcount != 1:
            raise DispatchError("Patient was already taken")
        conn.execute("COMMIT")             # leave critical section
```

```
    except Exception:
        conn.execute("ROLLBACK")
        raise
```

4.3 Bed manager module (counting semaphore)

```
def wait_bed(conn, visit_id):     # P(beds) - caller already holds the lock
    bed = conn.execute(
        "SELECT bed_id, label FROM beds WHERE status = 'free' "
        "ORDER BY bed_id LIMIT 1").fetchone()
    if bed is None:                                  # no bed: block
        conn.execute("UPDATE visits SET status = 'boarding' "
                     "WHERE visit_id = ?", (visit_id,))
        return None
    conn.execute("UPDATE beds SET status = 'occupied', visit_id = ? "
                 "WHERE bed_id = ? AND status = 'free'",
                 (visit_id, bed["bed_id"]))
    conn.execute("UPDATE visits SET status = 'admitted' "
                 "WHERE visit_id = ?", (visit_id,))
    return bed["label"]
```

signal_bed() does the opposite: when a bed is marked clean, it is given straight to the most urgent boarding patient, or set to free if nobody is waiting.

4.4 AI module and model architecture

Fig 5 shows the full AI pipeline. There are three models and two rule layers:

Table 2: Model architecture

Model	Algorithm	Inputs	Output	Settings
Triage level	Random forest classifier	15 features (8 numeric + 7 complaint flags)	Probability of ESI 1–5	200 trees, max depth 12, min leaf 2, balanced class weights
Consultation time	Random forest regressor	Same 15 features + final ESI	Minutes	150 trees, max depth 10, min leaf 4
Waiting time	Gradient boosting regressor	Replay estimate, ESI, patients ahead, doctors busy / on duty, arrivals last hour, hour	Minutes	300 trees, depth 3, learning rate 0.05, absolute error loss
Safety layer	Rules	Model probabilities	Final suggestion	Move up one level if P ≥ 0.30
Red flags	Rules	Vitals	ESI 1	SpO2 < 85, SBP < 80, AVPU ≥ P, HR > 150 or < 40, RR ≥ 35 or ≤ 8

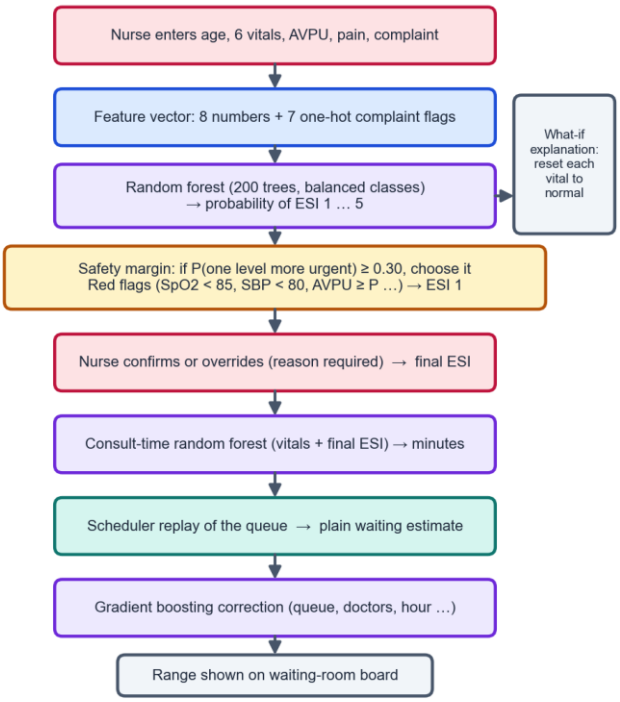


Fig 5: AI pipeline from vital signs to the waiting room display

The explanation shown to the nurse is a what-if analysis. Each abnormal input is set back to a normal value and the change in predicted urgency is measured. All these variants are sent to the model in one batch.

4.5 Web application module

Table 3: Main routes

Route	Who	What it does
/login, /logout	All	Login with hashed passwords
/triage, /api/triage/preview	Nurse	Triage form, live AI suggestion, issue token
/visits/<id>/retriage, /visits/<id>/left	Nurse	Re-triage, left without being seen
/doctor, /doctor/call-next, /doctor/preempt	Doctor	Console, dispatcher, ESI 1 preemption
/doctor/discharge, /doctor/admit	Doctor	Finish consultation (admit uses wait_bed)
/beds, /beds/<id>/release, /beds/<id>/clean	Staff	Bed board, signal_bed on clean
/board, /api/board	Public	Waiting room display without names
/admin, /admin/settings, /admin/race-test	Admin	Reports, aging interval, concurrency test
/api/state	Staff	Queue, doctors, beds, stats and log (polled every 5 s)

5. Technologies / Tools Used

Table 4: Technology stack

Layer	Technology	Version	Used for
Language	Python	3.12	Backend, OS modules, AI,

			tests
Web framework	Flask (with Jinja2, Werkzeug)	3.1.3	Routes, sessions, templates, password hashing
Database	SQLite (WAL mode)	3.x	All application data
Machine learning	scikit-learn	1.8.0	Random forest, gradient boosting
Numerical	NumPy, SciPy	2.4.4, 1.17.1	Arrays, dataset generation
Model storage	joblib	1.5.3	Saving / loading models
Frontend	HTML5, CSS3, JavaScript	–	Screens, live refresh
Testing	unittest, threading	standard library	Functional and concurrency tests
Version control	Git, GitHub	–	Source code
Deployment	Vercel (Python serverless)	–	Live demo
Report tools	Matplotlib, Playwright	–	Graphs and screenshots

Table 5: Programming languages

Language	**Where used**	**Approx. lines**
Python	Backend, scheduler, synchronization, AI, simulation, tests	2040
SQL	Schema, constraints, indexes, view, queries	95 + inline
JavaScript	Live refresh, triage preview, charts	520
HTML (Jinja2)	7 templates	400
CSS	Styling, light / dark theme	790

5.1 Configuration details

```
requirements.txt:  flask==3.1.3  scikit-learn==1.8.0  numpy==2.4.4
                   scipy==1.17.1  joblib==1.5.3
.python-version :  3.12

Environment variables
  MEDIQUEUE_DB   path of the SQLite file (tests use a temp file)
  VERCEL         set by Vercel -> database and results go to /tmp
  SECRET_KEY     Flask session key

SQLite:  journal_mode = WAL, busy_timeout = 15000 ms,
         foreign_keys = ON, isolation_level = None (manual BEGIN)
App:     aging interval 30 min (admin can change), aging cap = 3,
         ESI targets 0/10/30/60/120 min, safety margin 0.30,
         refresh every 5 s, clock fixed to IST (UTC+5:30), 8 beds
Demo logins (password demo123): priya, arun (nurse);
         rao, mehta, iyer, khan (doctor); admin
```

Versions are pinned because the saved model files have to be loaded with the same scikit-learn version that created them.

6. Dataset Description

Real emergency department records are not publicly available without approval. MIMIC-IV-ED [9] needs credentialed access through PhysioNet, which we could not complete during the semester, so we generated a synthetic dataset based on clinical rules.

Table 6: Dataset summary

Item	Details
Number of cases	3000
Case types	Minor injury 24%, fever 18%, abdominal pain 15%, trauma 14%, breathing difficulty 12%, chest pain 11%, stroke 6%
Features	Age, heart rate, systolic BP, respiratory rate, SpO2, temperature, consciousness (AVPU 0–3), pain (0–10), complaint
Label 1	ESI level 1–5 (from NEWS2 score + ESI decision points)
Label 2	Consultation time in minutes (from complaint, level and age)
Class counts	ESI 1: 292, ESI 2: 838, ESI 3: 736, ESI 4: 667, ESI 5: 467
Label noise	About 6% of levels shifted by one (to copy nurse disagreement)
Mean consultation time	31.6 minutes
Shift data	12 hour shifts (8 am – 8 pm), Poisson arrivals, more patients late morning and evening
Waiting-time data	6342 patients from 80 shifts for training, 1950 from 25 other shifts for testing

The ESI rules used for labelling are simple: unconscious or responding only to pain, SpO2 below 86, systolic BP below 80 or heart rate above 150 gives ESI 1; a NEWS2 score of 7 or more, stroke symptoms, chest pain above 40 years or pain 9–10 gives ESI 2; and so on down to ESI 5.

7. Data Preprocessing

- Clipping: every generated value is clipped to a realistic range (for example SpO2 70–100, heart rate 35–190, temperature 34–42 °C) so that no impossible values reach the model.
- Encoding: the chief complaint is one-hot encoded into 7 columns. Consciousness (AVPU) is converted to an ordinal number 0–3 because the levels have a natural order.
- No scaling: random forests and gradient boosting do not need feature scaling, so numeric values are used as they are.
- Class imbalance: ESI 1 has fewer cases (292), so the triage forest uses balanced class weights and the folds are stratified.

- Extra feature: the consultation time model also gets the final ESI level, because sicker patients take longer.
- Waiting-time features: for every simulated arrival we record the scheduler's own estimate and the queue state at that moment, and the actual wait as the target.
- Input validation in the app: the form and the database have range checks (for example SpO2 50–100, pain 0–10) so wrong entries are rejected before prediction.

8. Experimental Setup

Table 7: Hardware and software

Item	Details
Processor	13th Gen Intel Core i7-13620H
RAM	16 GB
Operating system	Windows 11 Home
Python	3.12.5
Libraries	scikit-learn 1.8.0, NumPy 2.4.4, Flask 3.1.3
Random seeds	42 (models), 7 (simulation)

8.1 Training and testing methodology

- Triage and consultation time models: 5-fold cross validation (stratified for the triage model). Every case is predicted once by a model that did not see it during training. The final models are then trained on all 3000 cases.
- Waiting-time model: trained on 80 simulated shifts and tested on 25 different shifts with another random seed, so the test patients and days are completely new to the model.
- Scheduling: 40 simulated 12 hour shifts with 4 doctors. All methods get exactly the same patients. The scheduler only sees the AI's level and predicted time, but the results are measured using the true level and true consultation time.
- Aging interval: 25 shifts for each value 5, 10, 15, 30, 60 minutes and no aging.
- Concurrency: 6 threads press "call next" together with 6 patients waiting, and 8 threads try to admit when only 3 beds are free. Each scenario is repeated 30 times, with a random 0.5–4 ms delay between read and write.

8.2 Baseline methods

Table 8: Baselines and proposed methods

Part	Baseline method(s)	Proposed method
Triage	NEWS2 score converted to a level; plain random forest	Random forest + safety margin + red flags
Scheduling	FCFS (token system), SJF on predicted time, priority only	Priority + capped aging + ESI 1 preemption
Waiting time	Scheduler replay only	Replay + gradient boosting correction
Synchronization	No lock (read, then write)	Critical section with BEGIN IMMEDIATE and status check

8.3 Performance metrics

- Triage: accuracy, macro F1, under-triage % (predicted less urgent than true), over-triage %, accuracy within ±1 level.
- Consultation and waiting time: mean absolute error (MAE) in minutes, % within 5 or 10 minutes.
- Scheduling: mean wait of ESI 1–2 patients, % seen within the ESI target time, 95th percentile wait of ESI 4–5 (starvation), worst wait per shift, mean wait of all patients, preemptions per shift.
- Synchronization: number of double assignments, average time per request.

9. Initial Implementation Screenshots

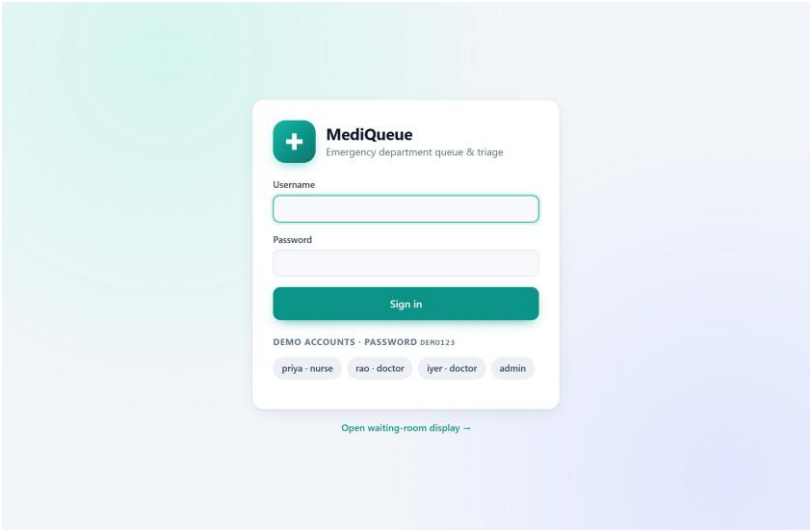


Fig 6: Login page

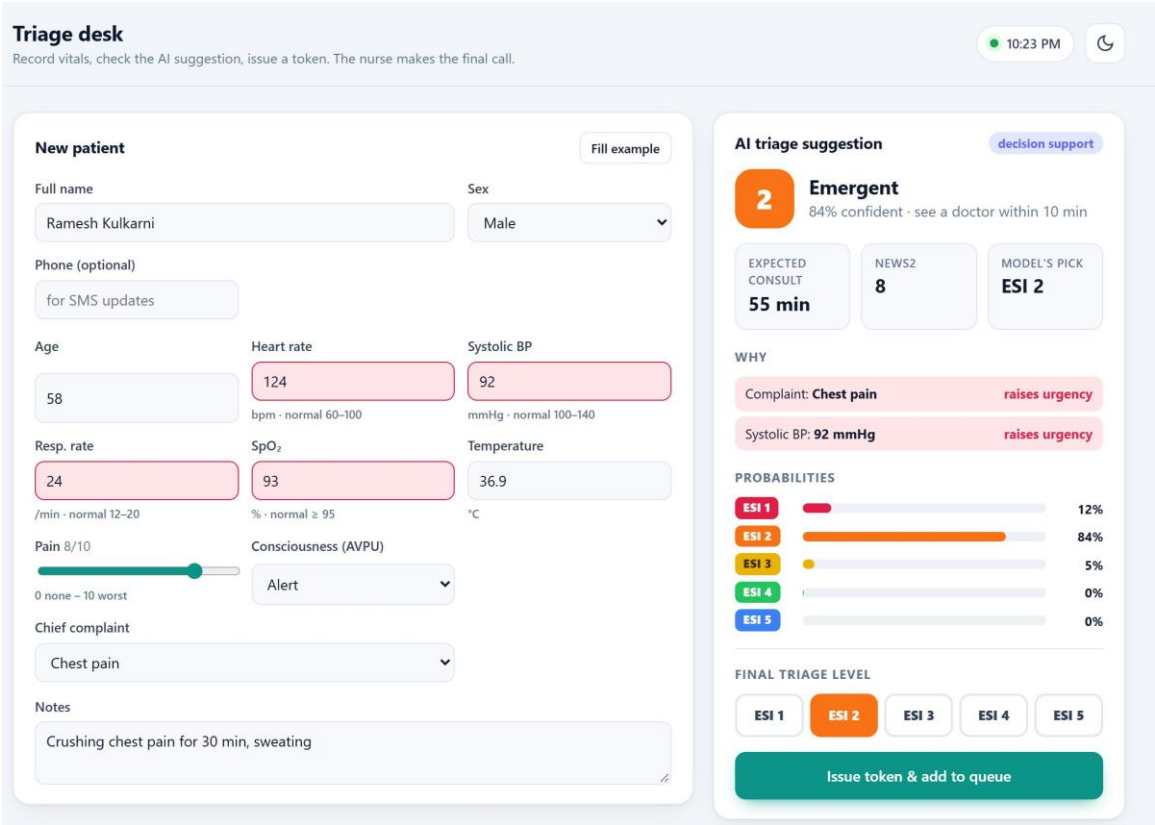


Fig 7: Triage desk with live AI suggestion and reasons

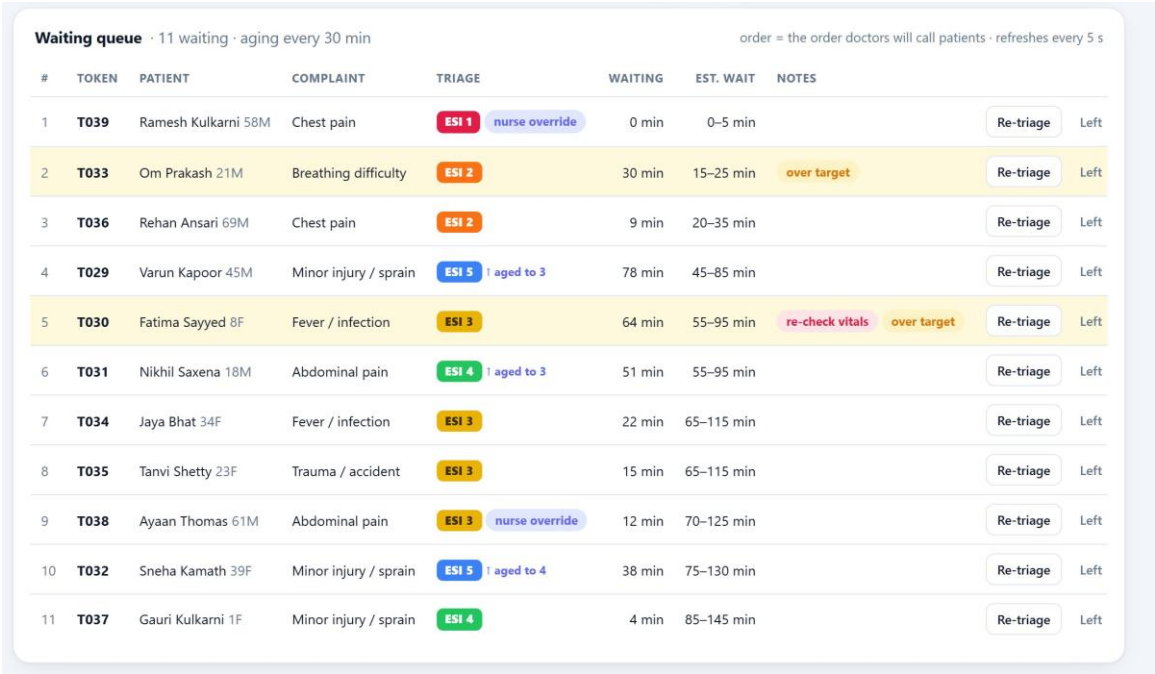


Fig 8: Waiting queue in dispatch order (aging and nurse override visible)

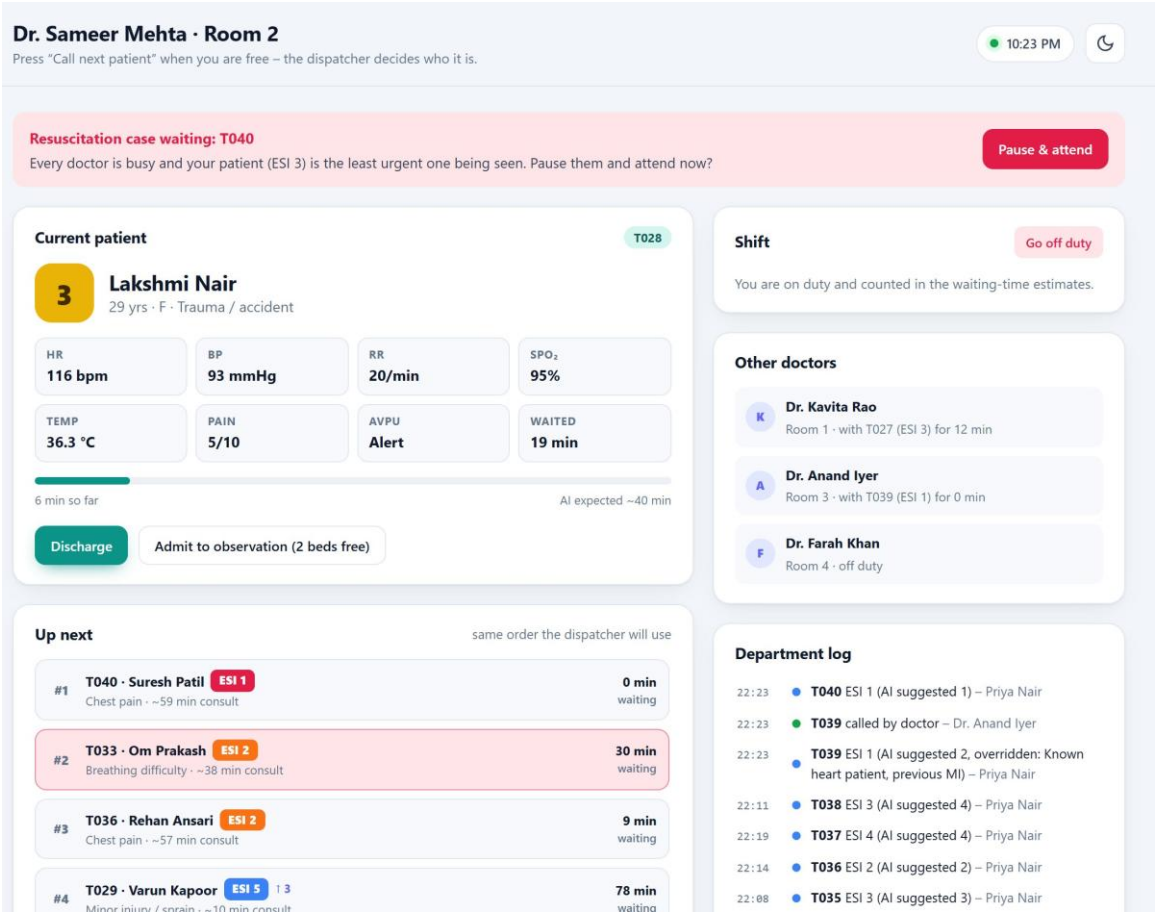


Fig 9: Doctor console asking to pause an ESI 3 consultation for an ESI 1 patient

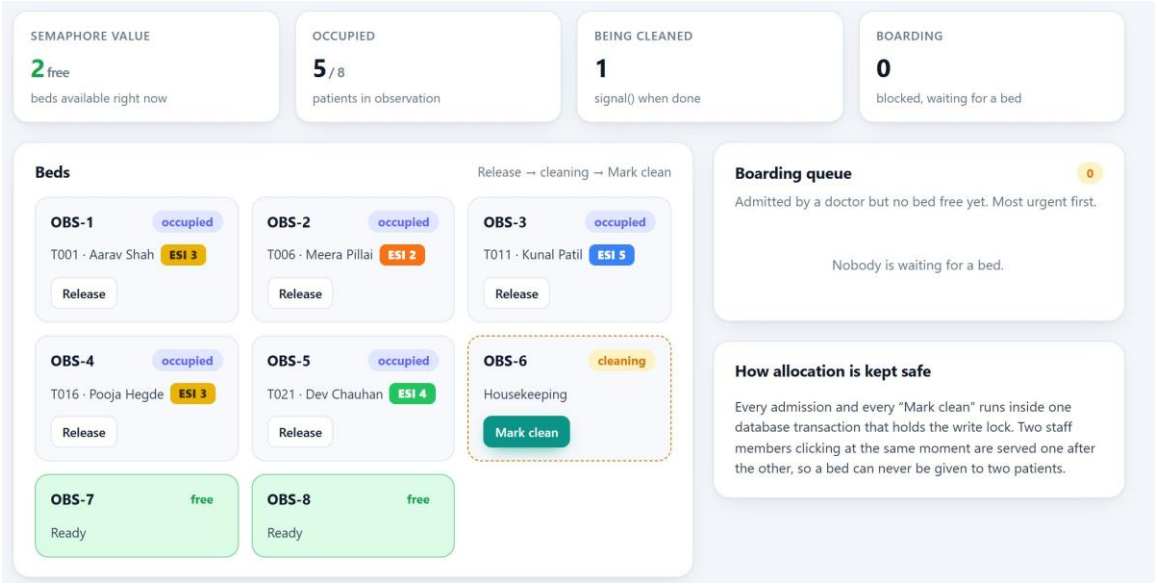


Fig 10: Bed board with semaphore value and boarding queue

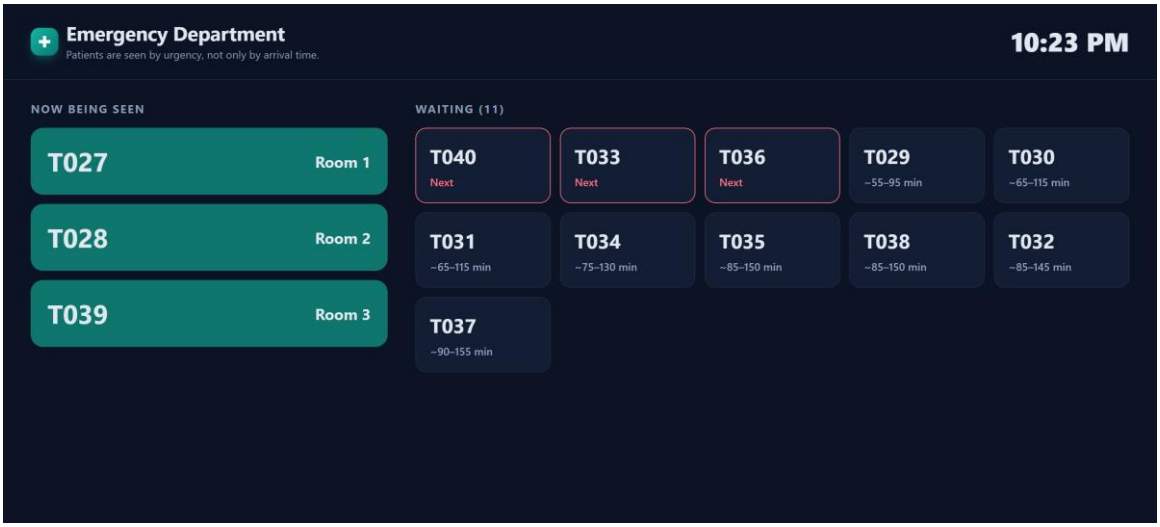


Fig 11: Waiting room display (tokens and waiting time, no names)

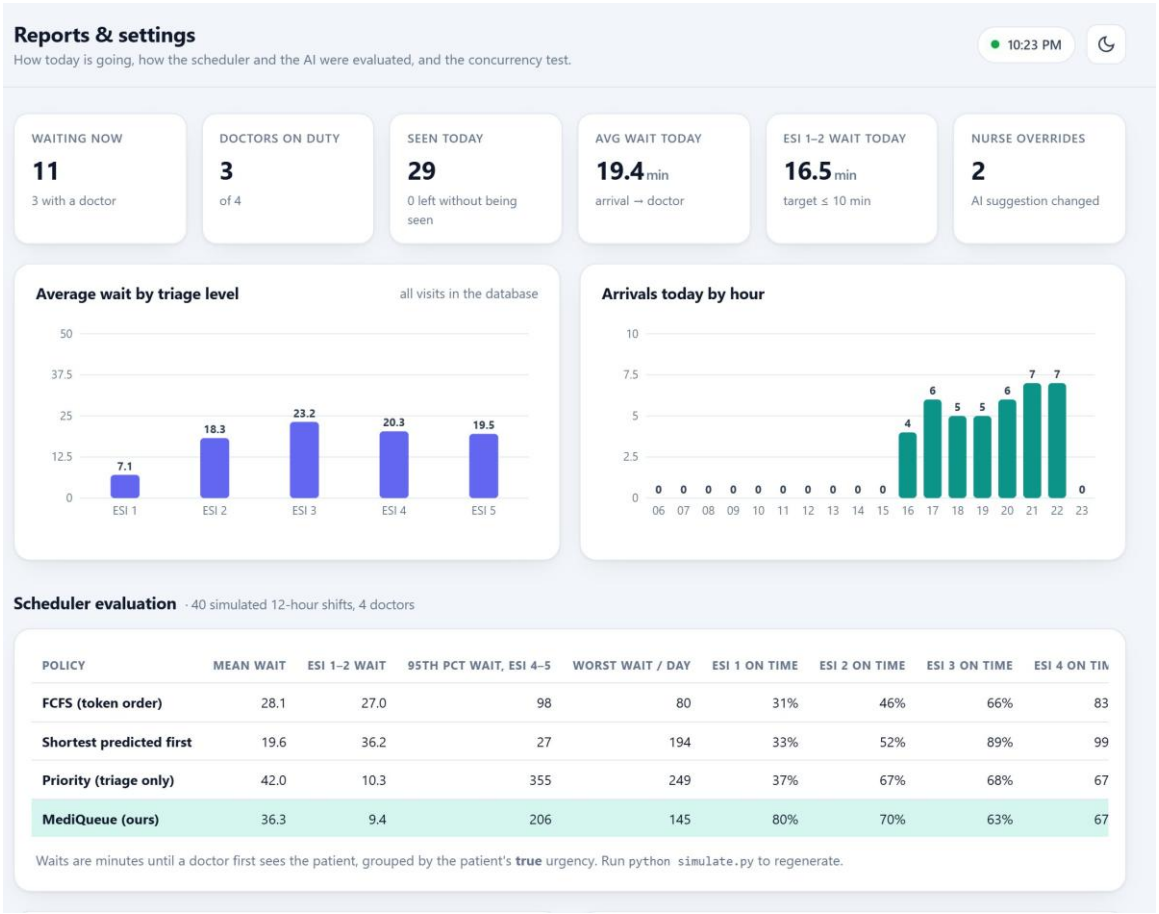


Fig 12: Admin reports page with the scheduler evaluation

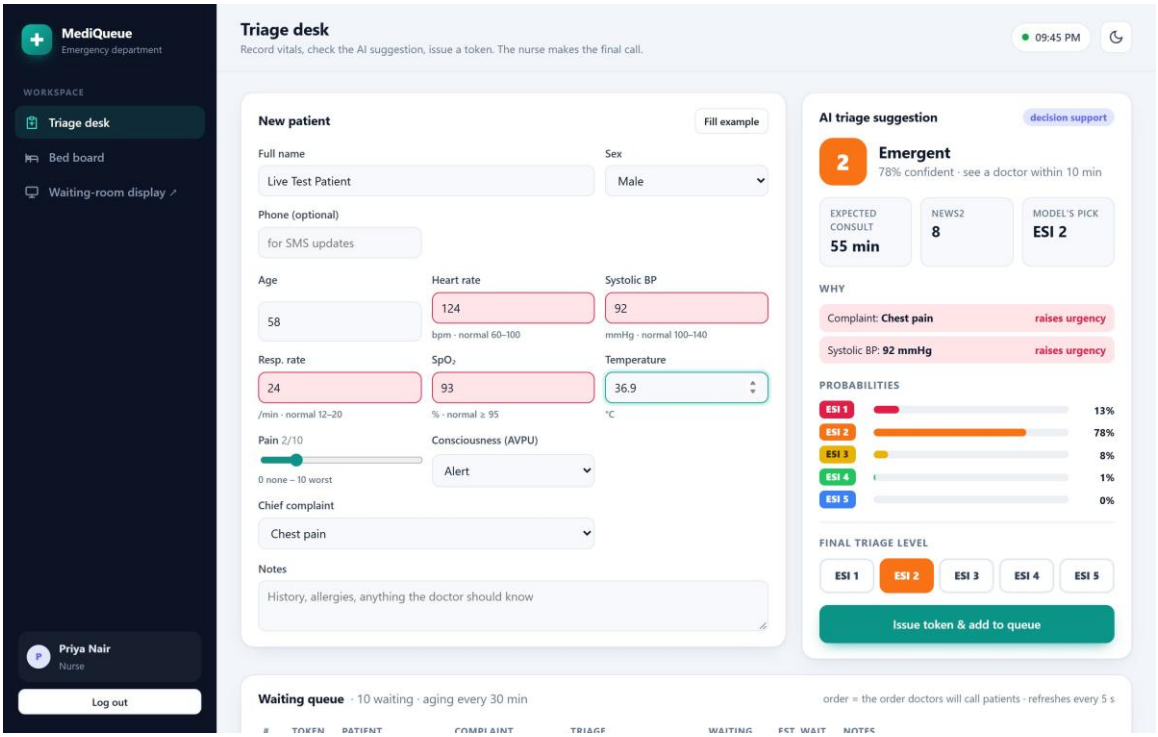


Fig 13: Same triage screen running on the live Vercel deployment

10. Preliminary Results

10.1 Triage model

Table 9: Triage results (5-fold cross validation, 3000 cases)

Method	Accuracy	Macro F1	Under-triage	Over-triage
NEWS2 only (baseline)	55.3%	0.543	41.5%	3.2%
Random forest (baseline)	86.2%	0.868	8.8%	5.0%
Random forest + safety rules (proposed)	82.8%	0.837	5.2%	12.0%

The plain random forest is the most accurate, but it under-triages almost 9% of patients. With the safety rules under-triage comes down to 5.2%. Accuracy drops a little because more patients are over-triaged, but in an emergency department seeing someone a bit earlier is much safer than making a sick patient wait. NEWS2 alone is poor because it does not use the complaint or pain. The plain forest is within one level of the true level 99.1% of the time.

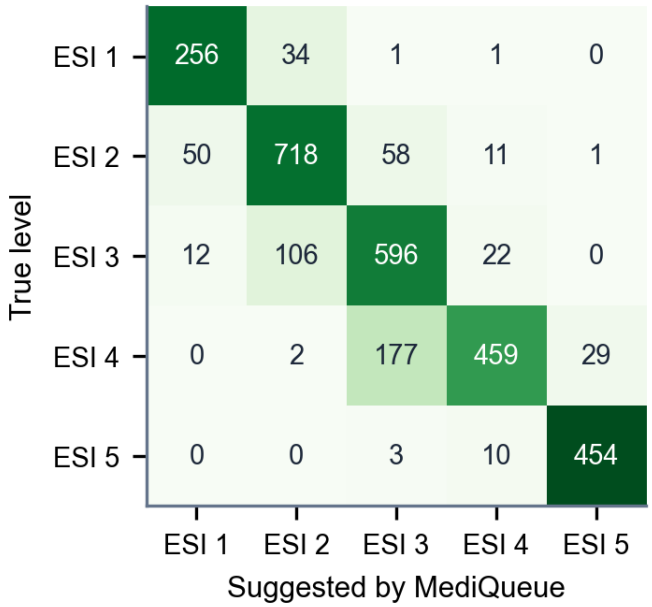


Fig 14: Confusion matrix of the proposed triage suggestion

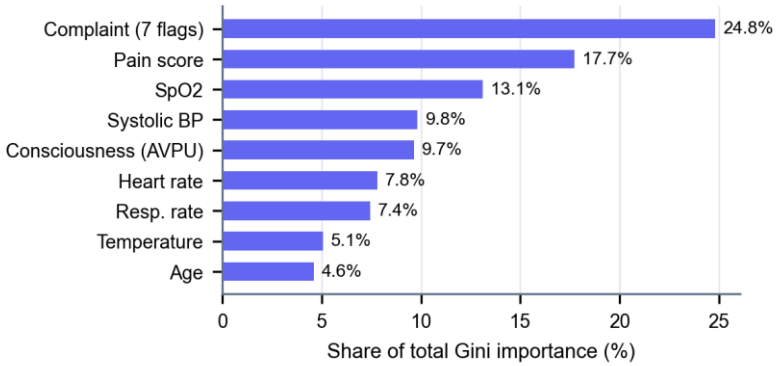


Fig 15: Feature importance of the triage model

The consultation time model has a mean absolute error of 4.0 minutes (average consultation 31.6 minutes) and 71% of its predictions are within 5 minutes. One suggestion including the explanation takes about 16 ms.

10.2 Scheduling methods

Table 10: Scheduling results over 40 simulated shifts (3174 patients, times in minutes)

Method	**Mean wait**	**ESI 1–2 wait**	**95th pct ESI 4–5**	**Worst wait / shift**	**ESI 1 on time**	**Preemptions / shift**
FCFS (baseline)	28.1	27.0	98	80	31%	0
SJF predicted (baseline)	19.6	36.2	27	194	33%	0
Priority only (baseline)	42.0	10.3	355	249	37%	0
MediQueue (proposed)	36.3	9.4	206	145	80%	4.2

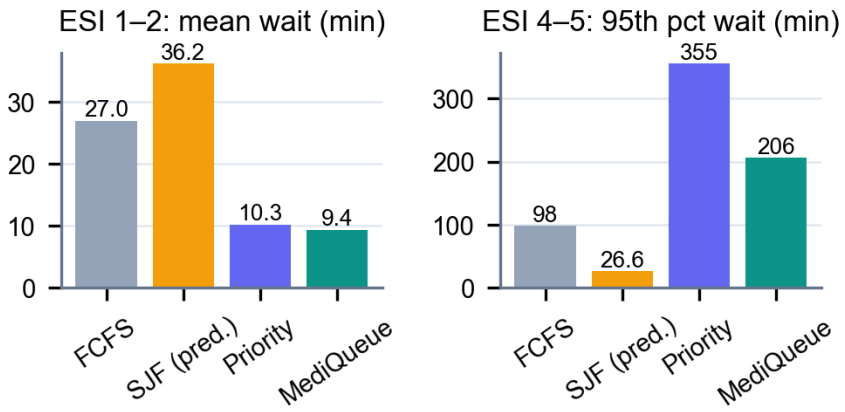


Fig 16: Wait of critical patients and 95th percentile wait of low priority patients

MediQueue gives the lowest waiting time for ESI 1–2 patients (9.4 min against 27.0 for FCFS) and sees 80% of ESI 1 patients immediately, compared to 37% with plain priority scheduling, because of preemption. It also cuts the 95th percentile wait of low priority patients from 355 to 206 minutes, so starvation is much less than with plain priority. The average wait of all patients is higher than FCFS and SJF. SJF has the lowest average only because it keeps short minor cases first and makes the serious (longer) cases wait, which is not acceptable here.

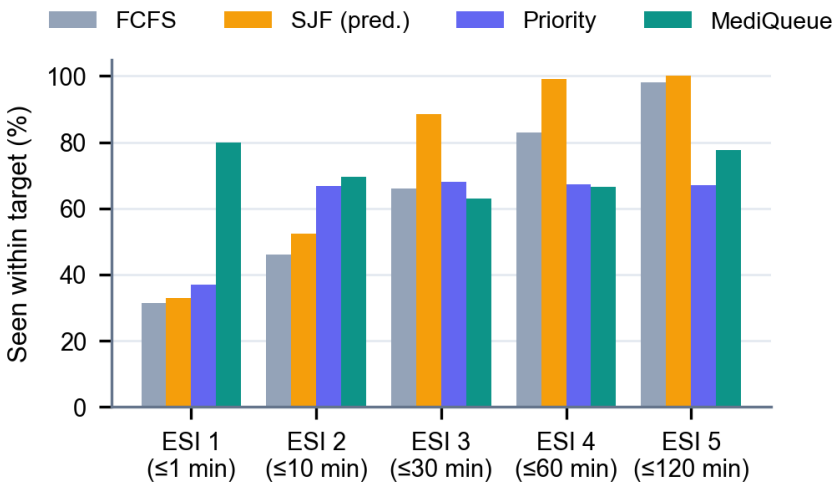


Fig 17: Percentage of patients seen within the ESI target time

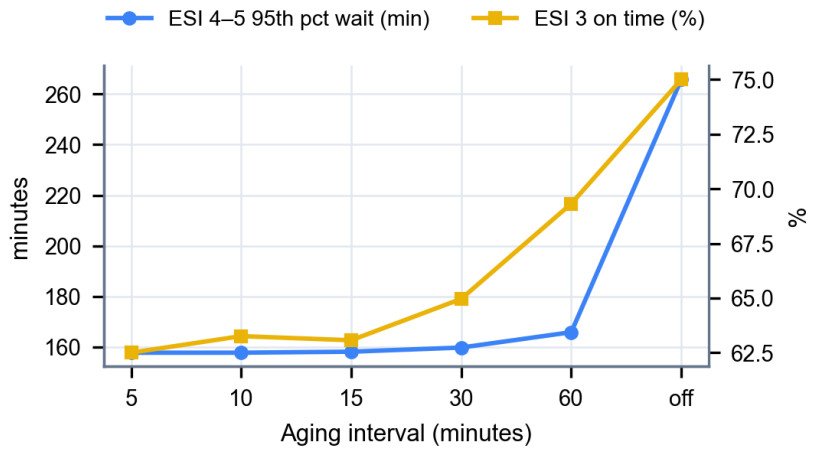


Fig 18: Effect of the aging interval

The aging sweep shows why we used 30 minutes. Up to 30 minutes the tail wait of low priority patients stays almost the same (158–160 min) while ESI 3 patients are seen on time more often (62% to 65%). After 30 minutes the tail starts growing, and with no aging it jumps to 266 minutes.

10.3 Waiting-time estimate

Table 11: Waiting-time estimate on 1950 test patients

Method	**MAE (min)**	**Within 10 min**	**MAE ESI 1–2**	**MAE ESI 3–5**
Scheduler replay only (baseline)	13.3	75%	≈3	17–22
Replay + gradient boosting (proposed)	8.9	77%	≈2	12–15

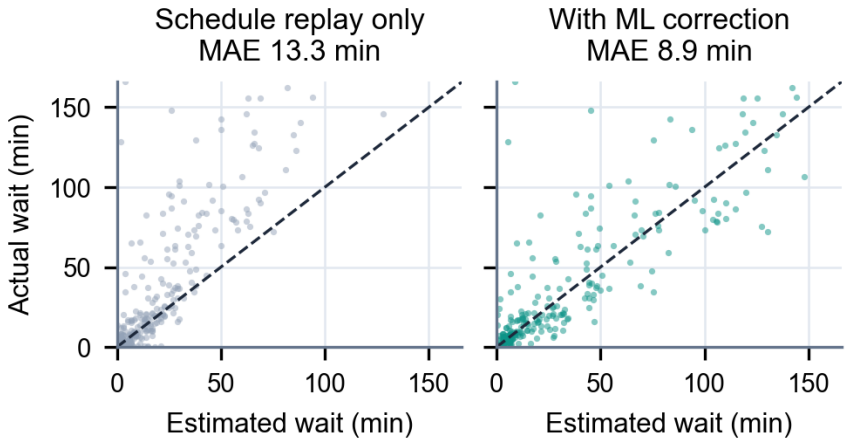


Fig 19: Estimated against actual waiting time

10.4 Synchronization

Table 12: Concurrency stress test (30 trials per scenario)

Scenario	Without lock	With critical section	Avg time per request
6 doctors press call next together	45 double assignments	0	22.0 ms → 34.5 ms
8 admissions, 3 free beds	50 beds given twice	0	30.3 ms → 48.1 ms

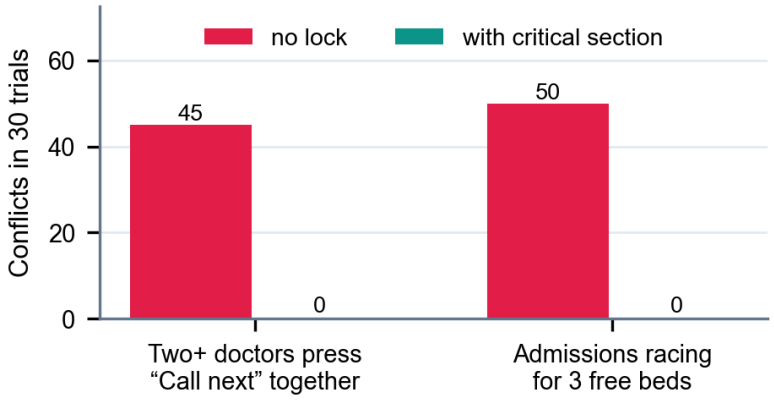


Fig 20: Conflicts with and without the critical section

All 14 functional tests pass, including the test where four doctors call the next patient at the same moment and each gets a different patient.

10.5 Comparison with existing methods

Table 13: Comparison with existing approaches

Approach	What it does	Limitation	MediQueue
Token / FCFS system	Calls in arrival order	Critical patients waited 27.0 min in our simulation	9.4 min for ESI 1–2
Plain priority triage	Most urgent first	Low priority tail wait 355 min (starvation)	206 min with capped aging
NEWS2 score [2]	Vital-sign score	55.3% accuracy, 41.5% under-triage on our data	82.8%, 5.2% under-triage
ML triage (Levin [6], Hong [7], Raita [8])	Predict urgency / outcome	No calling order, no waiting time, large private datasets	Prediction is used by the scheduler and shown with reasons
Wait prediction (Ang [11])	Queue features + Lasso	Separate from the dispatch rule	Built on a replay of our own scheduler

Our numbers cannot be compared directly with the published accuracy of [6]–[8] because they used real hospital data and we used synthetic data. The comparison above is about what each approach covers.

11. Problems Encountered and Solutions

Table 14: Problems and solutions

Problem	Solution
No access to real triage data (MIMIC-IV-ED needs credentialing).	Wrote a synthetic data generator based on NEWS2 and ESI rules; the code can switch to real data later.
The plain model under-triaged 8.8% of patients.	Added a safety margin (move up a level at $P \geq 0.30$) and red-flag rules; under-triage fell to 5.2%.
In our first version aging could go up to level 2. Under heavy load almost everyone reached level 2 and the queue behaved like FCFS.	Capped aging at level 3 so levels 1–2 stay for really serious patients.
We did not know which aging interval to use.	Ran a sweep (5–60 min) and picked 30 min.
Two doctors pressing the button together could get the same patient.	BEGIN IMMEDIATE + conditional UPDATE; the stress test shows 0 conflicts.
The simple waiting-time estimate was too optimistic for low priority patients.	Added the gradient boosting correction; error fell from 13.3 to 8.9 min.
The explanation called the model about 10 times, so each suggestion took about 52 ms.	Sent all variants in one batch and used one thread per prediction; now about 16 ms.
Readers were blocked while a doctor's request held the write lock.	Switched SQLite to WAL mode and set busy_timeout to 15 s.
Vercel's file system is read-only.	The demo database is created and seeded in /tmp when the server starts.
Times on Vercel were in UTC (5.5 hours behind).	Used a fixed IST clock in the code instead of the server time.
On Vercel, two requests from one page could reach different server instances with different data.	Sent the doctor's patient inside the same /api/state response. For multi-screen demos we run it locally; a shared hosted database would remove this completely.

Source Code and Deployment

Table 15: Links

Item	Link
GitHub repository (public)	https://github.com/anandaahan7-prog/mediqueue
Live deployment (Vercel)	https://mediqueue-eight-tan.vercel.app
Demo login	priya / demo123 (nurse), iyer / demo123 (doctor), admin / demo123

```
git clone https://github.com/anandaahan7-prog/mediqueue.git
cd mediqueue
pip install -r requirements.txt
python seed.py           # demo data
python app.py            # http://localhost:5000
python -m unittest tests.test_system -v
```

12. Updated References

[1] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4, Implementation Handbook. Rockville, MD: AHRQ, Publication No. 12-0014, 2012.

[2] Royal College of Physicians, National Early Warning Score (NEWS) 2: Standardising the Assessment of Acute-Illness Severity in the NHS. London: RCP, 2017.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[4] E. W. Dijkstra, "Cooperating sequential processes," Technological University, Eindhoven, Tech. Rep. EWD-123, 1965.

[5] L. Kleinrock, Queueing Systems, Volume II: Computer Applications. New York: Wiley, 1976.

[6] S. Levin et al., "Machine-learning-based electronic triage more accurately differentiates patients with respect to clinical outcomes compared with the Emergency Severity Index," Ann. Emerg. Med., vol. 71, no. 5, pp. 565-574, 2018.

[7] W. S. Hong, A. D. Haimovich, and R. A. Taylor, "Predicting hospital admission at emergency department triage using machine learning," PLoS ONE, vol. 13, no. 7, e0201016, 2018.

[8] Y. Raita, T. Goto, M. K. Faridi, D. F. M. Brown, C. A. Camargo, and K. Hasegawa, "Emergency department triage prediction of clinical outcomes using machine learning models," Crit. Care, vol. 23, art. 64, 2019.

[9] A. Johnson, L. Bulgarelli, T. Pollard, L. A. Celi, R. Mark, and S. Horng, "MIMIC-IV-ED (version 2.2)," PhysioNet, 2023.

[10] G. B. Smith, D. R. Prytherch, P. Meredith, P. E. Schmidt, and P. I. Featherstone, "The ability of the National Early Warning Score (NEWS) to discriminate patients at risk of early cardiac arrest, unanticipated intensive care unit admission, and death," Resuscitation, vol. 84, no. 4, pp. 465-470, 2013.

[11] E. Ang, S. Kwasnick, M. Bayati, E. L. Plambeck, and M. Aratow, "Accurate emergency department wait time prediction," Manuf. Serv. Oper. Manag., vol. 18, no. 1, pp. 141-156, 2016.

[12] S. Saghafian, W. J. Hopp, M. P. Van Oyen, J. S. Desmond, and S. L. Kronick, "Complexity-augmented triage: A tool for improving patient safety and operational efficiency," Manuf. Serv. Oper. Manag., vol. 16, no. 3, pp. 329-345, 2014.

[13] S. Saghafian, G. Austin, and S. J. Traub, "Operations research/management contributions to emergency department patient flow optimization: Review and research prospects," IIE Trans. Healthc. Syst. Eng., vol. 5, no. 2, pp. 101-123, 2015.

[14] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135-1144.

[15] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, 2001.

[17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[18] SQLite Consortium, "File locking and concurrency in SQLite version 3." [Online]. Available: https://www.sqlite.org/lockingv3.html

[19] Pallets Projects, "Flask documentation (3.1.x)." [Online]. Available: https://flask.palletsprojects.com

[20] Vercel, "Using the Python runtime with Vercel Functions." [Online]. Available: https://vercel.com/docs/functions/runtimes/python